

Failure-Aware Enhancements for Large Language Model (LLM) Code Generation: An Empirical Study on Decision Framework

Jianru Shen*, Zedong Peng*, Lucy Owen

University of Montana, Missoula, MT, USA

jianru.shen@umconnect.umt.edu, zedong.peng@mso.umt.edu, lucy.owen@mso.umt.edu

Abstract—Large language models (LLMs) show promise for automating software development by translating requirements into code. However, even advanced prompting workflows like progressive prompting often leave some requirements unmet. Although methods such as self-critique, multi-model collaboration, and retrieval-augmented generation (RAG) have been proposed to address these gaps, developers lack clear guidance on when to use each. In an empirical study of 25 GitHub projects, we found that progressive prompting achieves 96.9% average task completion, significantly outperforming direct prompting (80.5%, Cohen’s $d=1.63$, $p < 0.001$) but still leaving 8 projects incomplete. For 6 of the most representative projects, we evaluated each enhancement strategy across 4 failure types. Our results reveal that method effectiveness depends critically on failure characteristics: Self-Critique succeeds on code-reviewable logic errors but fails completely on external service integration (0% improvement), while RAG achieves highest completion across all failure types with superior efficiency. Based on these findings, we propose a decision framework that maps each failure pattern to the most suitable enhancement method, giving practitioners practical, data-driven guidance instead of trial-and-error.

Index Terms—Software Engineering, Large Language models (LLMs), RAG, Failure Taxonomy, Code Generation

I. INTRODUCTION

Imagine a software engineer using an AI pair programmer to implement a new web service. Initially, the large language model (LLM) assistant (e.g., GitHub Copilot or ChatGPT) produces promising code for basic CRUD functionalities. The engineer follows a progressive prompting workflow: first prompting for requirements analysis, then architectural design, test specification, and finally code implementation, which is a strategy that generally yields better results than one-shot code generation [1]. Yet, as the project progresses, the assistant struggles with certain tasks. For example, when integrating an external payment API, the generated code omits essential steps and fails to fully meet the requirement. The developer is left wondering: should they prompt the LLM to critique and refine its output, fetch relevant API documentation for the model, or involve a different model altogether? This scenario highlights a real-world challenge: even the best prompting practices can systematically fail on complex requirements, and it is unclear which remedial strategy to apply for a given failure.

Progressive prompting still misses some requirements. In our baseline study of 25 GitHub projects, it significantly outperformed direct prompting in coverage, but some projects remained incompletely implemented. We found that 8 out of 25 projects (32%) remained incomplete. Prior work has proposed a variety of enhancement strategies to close these gaps [2]–[5], but these techniques have not been systematically organized or compared by failure mode. In this paper, we synthesize the literature into three representative enhancement families and instantiate each with a concrete method. The three enhancement methods we study are not merely abstract categories: in this work, they are concretely instantiated, tuned, and integrated by us into practical pipelines for LLM-based development. At the end, we provide structured guidance on **when** each family helps, **which** failure patterns it mitigates, and **where** gaps remain.

The main contribution of this work is a failure-aware enhancement framework for LLM-based code generation that goes beyond standard progressive prompting. We design and implement three concrete enhancement pipelines—Self-Critique, Multi-Model Collaboration, and Retrieval-Augmented Generation (RAG) assisted, and then derive a taxonomy that links failure types to the strengths and limitations of each pipeline. Our study yields a failure-driven decision framework with quantified time–efficiency trade-offs, giving practitioners actionable guidance on when and how to apply each enhancement method. In what follows, we present the related work in Section II, the methodology in Section III, and the empirical evaluations in Section IV. Section V discusses our decision framework, Section VI outlines the threats to validity, and finally, Section VII concludes the paper.

II. BACKGROUND AND RELATED WORK

Large language models have demonstrated remarkable capabilities in automated code generation, leveraging advanced reasoning techniques to translate natural language requirements into functional implementations. Chain-of-thought (CoT) prompting, introduced by Wei et al. [6], decomposes complex reasoning into intermediate steps, significantly improving LLM performance on mathematical and logical tasks. Building upon CoT, Wang et al. proposed self-consistency reasoning paths achieving substantial accuracy gains on arith-

*Corresponding authors

metric benchmarks [7]. Progressive prompting extends these ideas to software development by structuring code generation through sequential phases: requirements analysis, architectural design, and implementation, which is analogous to traditional software engineering workflows [1].

Researchers have proposed complementary approaches: RAG augments LLMs with external documentation through differentiable retrieval mechanisms [8], while Tree-of-Thoughts (ToT), proposed by Yao et al. [9], enables deliberate exploration of multiple solution paths via search algorithms. Yao et al.’s work on multiple choice question generation, with iterative self-critique, correction, and comparison feedback, demonstrates how LLMs can improve outputs through self-review cycles [2], [10]. Multi-model collaboration strategies [3], [4], such as those studied by Dong et al. on efficient question-answering with strategic multi-model collaboration on knowledge graphs, leverage complementary strengths of different models for better performance [3]. Recent surveys document the rapid evolution of LLM-based code generation agents that incorporate planning, iterative refinement, and multi-model collaboration [11]. However, practitioners lack systematic guidance on which enhancement method to apply for specific failure types, motivating our empirical investigation of failure-aware selection strategies.

III. METHODOLOGY

We conducted a two-phase empirical study to evaluate enhancement strategies for LLM-based code generation. Phase 1 established a baseline by evaluating progressive prompting on 25 open-source projects. Phase 2 investigated three enhancement strategies on 6 challenge projects where baseline progressive prompting failed to achieve complete requirement coverage.

A. Phase 1: Baseline Establishment

Project Selection. We selected 25 open-source GitHub projects using three criteria: (1) detailed step-by-step implementation instructions in README files, (2) clear project goals and requirements, and (3) numbered feature lists enabling objective task completion evaluation. Projects spanned diverse domains including e-commerce (online bookstores, shopping carts), booking systems (train tickets, hotels, car rentals), healthcare (pharmacy management), microservices (food delivery), search engines (Wikipedia IR), and DevOps infrastructure (containerized deployments). Programming languages included Java, Python, JavaScript, PHP, TypeScript, and Go, with project complexity ranging from 3 to 31 tasks (average: 13 tasks).

Evaluation Approach. We use original GitHub implementation serving as ground truth for requirement coverage. For each project, we compared two code-generation approaches using a state-of-the-art LLM (GPT-5) [12]: (a) Direct prompting: Code generation directly from requirements without intermediate design phases. Requirements were provided to the LLM with the prompt “Generate complete code for this project.” Multiple prompts were sometimes needed due to

output length limitations, but no structured workflow (e.g., design or planning phases) was employed, and (b) Progressive prompting: Multi-step approach following the sequence [1]: *requirements analysis* → *architectural design* → *test case specification* → *code implementation*.

B. Phase 2: Enhancement Strategy Evaluation

1) *Challenge Project Selection:* We focused on the projects where progressive prompting failed to complete all tasks. From the 8 incomplete projects, we selected 6 representative projects following a maximum diversity sampling strategy [13]. The two excluded projects exhibited redundant failure patterns already covered by our selected samples, shown in the table I. These 6 projects provide comprehensive coverage of observed failure patterns while maintaining experimental feasibility (6 projects × 3 methods = 18 experiments). For each project, we applied three enhancement strategies:

- **Self-Critique:** This method applies iterative self-review to baseline progressive prompting. After initial code generation, we prompted the LLM to critique its own output: “Review the generated code carefully. Is it complete? What requirements are missing or only partially implemented?” The LLM then identified gaps and generated fixes for incomplete implementations. This critique-fix cycle continued until the LLM reported all requirements complete or no further improvements were identified. Self-Critique uses the same model as baseline progressive prompting, adding only review iterations. The hypothesis is that explicit code review can identify incomplete implementations that baseline prompting missed.
- **Multi-Model Collaboration:** This method combines two different LLMs with complementary strengths: GPT-5 for requirements analysis and architectural design, and Claude Sonnet 4.5 for code implementation. The process has two phases: (1) **Design phase:** GPT-5 analyzes requirements and produces detailed architectural specifications including component structure, API design, data models, and integration patterns; (2) **Implementation phase:** The architectural specification is provided to Claude Sonnet 4.5, which generates code following the design. This separation leverages GPT-5’s advanced reasoning capabilities for system architecture while utilizing Claude’s code generation strengths. The hypothesis is that explicit architectural design reduces ambiguity that causes implementation gaps.
- **RAG-Assisted:** This method augments baseline progressive prompting with retrieved documentation and code examples. Using Claude Sonnet 4.5 with API access, we retrieved relevant materials for each project: (1) official framework documentation (e.g., Spring Boot documentation for P2, Django i18n guides for P25, RabbitMQ tutorials for P8), (2) similar open-source projects from GitHub (e.g., healthcare management systems for P18, microservices with message queues for P8), and (3) implementation patterns matching the project’s technology stack. Retrieved materials were provided as context during code generation with the prompt:

TABLE I
CHALLENGE PROJECT CHARACTERISTICS

ID	Project	Languages	Completion	Missing / incomplete functionality	Failure Type
P2	Train Ticket Reservation	Java/HTML/CSS	16/17 (94.1%)	Missing admin profile update endpoint	Local Logic Failures
P8	Food Delivery Microservices	Java/Shell	23/24 (95.8%)	Incomplete RabbitMQ message queue integration	External Integration Failures
P10	Shopping Cart	Java/HTML/CSS	20/22 (90.9%)	Missing inventory management and cart quantity logic	Local Logic Failures
P14	Book Store DevOps	Python/HTML/CSS/HCL/JS	29/31 (93.5%)	Incomplete Gunicorn multi-process metrics and VM provisioning workflow	Infrastructure Configuration Failures
P18	Pharmacy Management	Python/HTML/CSS/JS	10/11 (90.9%)	Missing patient medication self-management with healthcare business rules	Domain Knowledge Failures
P25	Expense Tracker	JS/TypeScript	12/13 (92.3%)	Incomplete multi-currency API integration and i18n framework setup	External Integration Failures

“Using these examples and documentation as reference, implement the following requirements...” The hypothesis is that concrete examples and authoritative documentation reduce implementation ambiguity and provide proven patterns for complex features.

2) *Evaluation Metrics and Process*: For each project-method combination (18 total experiments), we measured five metrics: *task completion rate*, *improvement over baseline*, *total LLM generation time*, *prompt count*, and *an efficiency metric (minutes per percentage-point (pp) improvement)*. The min/pp metric enables fair comparison across projects with different baseline completion rates and allows practitioners to assess cost-benefit trade-offs.

We assessed task completion through systematic manual evaluation. For each project, we compared generated code against requirement specifications extracted from GitHub README files. Each task was classified as: **Complete**: Fully implemented with all specified functionality; **Partial**: Implementation present but missing required features or edge cases; **Missing**: No implementation found in generated code.

To analyze enhancement methods from a *failure-aware* perspective, we classify each incomplete task not by low-level bug patterns [14], but by the kind of *missing functionality* in the target system. In our experiments, models failed less on basic syntax, type correctness, or straightforward control flow; most generated code compiled and followed common framework idioms without manual repair. Instead, the remaining errors clustered around high-level functional omissions [15], along four dimensions:

- Local Logic Failures: Issues solvable through code review (CRUD, validation rules)
- External Integration Failures: Requiring API docs, service configs
- Domain Knowledge Failures: Requiring specialized expertise
- Infrastructure Configuration Failures: Requiring deployment knowledge.

They correspond to missing pieces of an application’s functionality, not to specific coding mistakes. In that sense they are a high-level view of what’s “incomplete” in each project.

Classification was based on code inspection without execution testing, as projects require complex environment setup (databases, message queues, cloud resources, API keys). To

ensure objectivity, we created an evaluation checklist for each project listing all required tasks and their acceptance criteria before examining generated code.

Time measurements captured wall-clock time from first prompt submission to final code generation for each method, excluding time spent by the human evaluator formulating prompts or reviewing intermediate outputs. For Self-Critique, time included all critique-fix cycles. For Multi-Model, time included both GPT-5 design phase and Claude implementation phase. For RAG-Assisted, time included retrieval plus generation with retrieved context. All measurements represent LLM generation time only, not human thinking time.

IV. EVALUATION

This section evaluates baseline performance and our enhancements under three research questions. **RQ1**: How does progressive prompting compare to direct prompting in task completion? **RQ2**: How do Self-Critique, Multi-Model Collaboration, and RAG-Assisted perform on progressive prompting failures, and how does their effectiveness vary by failure type? **RQ3**: What are the time–efficiency trade-offs among these enhancement methods? We also discuss cost-effectiveness and present a practical decision framework for practitioners. Experimental materials are available at <https://doi.org/10.5281/zenodo.17637008>.

A. Baseline Performance (RQ1)

Baseline results (Table II) on 25 projects show progressive prompting averaged 96.9% completion (SD 5.8%) versus 80.5% (SD 13.0%) for direct prompting (Cohen’s $d = 1.63$, $p < 0.001$). SD denotes the standard deviation of project-level completion rates, and Cohen’s d is a standardized effect size where values above 0.8 are typically interpreted as large. Progressive prompting completed all tasks in 17/25 projects (68%) compared to 4/25 (16%) for direct prompting. Human-written solutions averaged 95.7% completion, indicating that progressive prompting nearly matches human coverage. These findings confirm that progressive prompting significantly outperforms direct prompting (RQ1).

However, 8 of the 25 projects remained incomplete (completion 90.9%–95.8%), comprising 19 missing tasks in four failure categories: Local Logic Failures (1 project), External Integration Failures (2 projects), Domain Knowledge Failures (2 projects), and Infrastructure Configuration Failures (1 project).

TABLE II
BASELINE PERFORMANCE COMPARISON (25 PROJECTS)

Method	Avg Compl	Median	SD	100% Rate	$\geq 90\%$ Rate	Avg Prompts
Human	95.7%	100%	—	16/25 (64%)	22/25 (88%)	—
Direct	80.5%	80%	13.0%	4/25 (16%)	6/25 (24%)	1.8
Progressive	96.9%	100%	5.8%	17/25 (68%)	24/25 (96%)	20.4

These gaps show that progressive prompting alone does not achieve full coverage, motivating the need for enhancement strategies.

B. Enhancement Strategy Effectiveness (RQ2)

Enhancement results (Table III) on six challenge projects show RAG-Assisted achieved the best performance 4/6 projects (67%) and Self-Critique 2/6 (33%), while Multi-Model not rank first in any project even though it reached 100% completion on 5/6 projects (83%). Average completion rates were 96.0% (Self-Critique, +3.0pp), 99.2% (Multi-Model, +6.3pp), and 99.2% (RAG-Assisted, +6.3pp).

Failure-type effects: Self-Critique solved P2 (logic failures) and P10 (domain Knowledge failures) with 100% completion, but made no improvement on P8, P18, and P25 (external/domain/integration failures). RAG-Assisted and Multi-Model succeeded on all projects except P10, where both stalled at 95.5% due to complex inventory edge cases requiring further refinement.

Integration tasks: On P8, P14, P18, and P25 (external APIs, infrastructure, and domain-specific logic), RAG-Assisted consistently reached 100% completion while Self-Critique made no progress. This highlights that external documentation and concrete examples are critical for resolving these failure types.

Code-reviewable failures: Projects P2 and P10 had logic or missing-function errors visible in the code. Self-Critique’s iterative review identified these gaps and fixed them, indicating it is most effective on failures that are “code-reviewable” without external knowledge.

Multi-Model and RAG-Assisted both averaged 99.2% completion, but assisted won 4/6 projects because it completed them faster with higher confidence. Multi-Model’s zero led in none reflect high reliability (5/6 projects at 100%) but longer runtimes, making it a secondary choice when RAG-Assisted also succeeds. Self-Critique’s 33% win rate (2/6) highlights its niche strength on code-reviewable failures and its limitations on integration/domain tasks.

A Friedman test [16] found no significant difference across methods ($\chi^2 = 3.60$, $df = 2$, $p = 0.165$), but large effect sizes (Cohen’s $d > 1.0$ for all methods vs baseline) imply practical differences. Overall, these results (RQ2) confirm that failure type, rather than method sophistication, determines effectiveness.

C. Cost-Benefit Analysis and Efficiency (RQ3)

Figure 1 shows the improvement vs. time for all 18 experiments. RAG-Assisted was most efficient (31.5 min per percentage point), compared to 40.6 for Self-Critique and 47.4 for

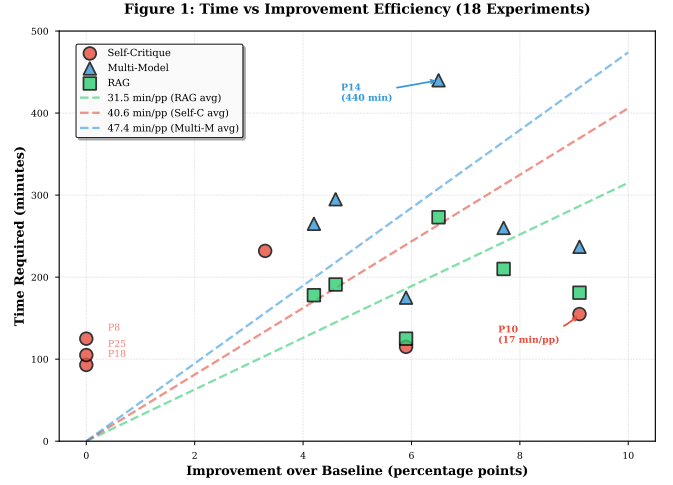


Fig. 1. Time vs Improvement Efficiency. RAG-Assisted (green squares); Self-Critique (red circles); Multi-Model (blue triangles)

Multi-Model. Efficiency differed significantly (Friedman test: $\chi^2 = 12.0$, $df = 2$, $p = 0.0025$; all pairwise $p = 0.031$). RAG-Assisted’s advantage comes from directly applying retrieved patterns, avoiding the trial-and-error of Multi-Model’s design first or Self-Critique’s iterative approaches. These efficiency results highlight three practical considerations when choosing an enhancement method:

Speed vs. Reliability: Self-Critique is fastest when it works (138 min avg) but fails on non-code-reviewable tasks. RAG-Assisted balances speed and reliability (193 min avg) across all tasks. Multi-Model is slowest (279 min) but most reliable (5/6 projects 100% completion), making it the choice when correctness is paramount.

Prompts vs. Quality: Multi-Model uses the most prompts (30 on average), RAG-Assisted about 27 (including retrieval), and Self-Critique 23. However, prompt count did not reflect solution quality: Self-Critique’s fewer prompts stem from its failure on complex tasks, not from efficiency.

Upfront Cost vs. Adaptation: RAG-Assisted needs an initial setup (API access, documentation) but then adapts efficiently to different failure types. Multi-Model needs no special setup but consistently incurs high time cost. Self-Critique requires no setup but can’t handle failures that aren’t visible in code.

Overall, RAG-Assisted offers the best cost-benefit (31.5 min/pp) while maintaining high reliability (99.2% completion), addressing RQ3.

V. DISCUSSION

A. Enhancement

Our results reveal that enhancement method effectiveness depends critically on failure characteristics rather than method sophistication.

Self-Critique Failed on External Service Integration. P8 (Food Delivery) required RabbitMQ integration. Self-Critique reviewed code but couldn’t identify missing queue configurations—these aren’t inferrable from application code alone. RAG-Assisted retrieved Spring Cloud RabbitMQ tutorials with complete patterns (exchanges, routing keys, listeners),

TABLE III
ENHANCEMENT RESULTS FOR SIX CHALLENGE PROJECTS

ID	Project	Tasks	Baseline	Self-C	Multi-M	RAG-A	Best	Improv
P2	Train Ticket	17	94.1%	100%	100%	100%	All three	+5.9pp
P8	Food Delivery	24	95.8%	95.8%	100%	100%	RAG-A (fastest)	+4.2pp
P10	Shopping Cart	22	90.9%	100%	95.5%	95.5%	Self-C	+9.1pp
P14	Book Store	31	93.5%	96.8%	100%	100%	RAG-A (fastest)	+6.5pp
P18	Pharmacy	11	90.9%	90.9%	100%	100%	RAG-A (fastest)	+9.1pp
P25	Expense Tracker	13	92.3%	92.3%	100%	100%	RAG-A (fastest)	+7.7pp
Average		19.7	92.9%	96.0%	99.2%	99.2%	–	7.1pp

TABLE IV
SUMMARY STATISTICS ACROSS ENHANCEMENT METHODS

Metric	Self-C	Multi-M	RAG-A
Avg Completion	96.0%	99.2%	99.2%
Improvement	+3.0pp	+6.3pp	+6.3pp
100% Projects	2/6 (33%)	5/6 (83%)	5/6 (83%)
Method rank first	2/6 (33%)	0/6 (0%)	4/6 (67%)
Avg Time	138 min	279 min	193 min
Avg Prompts	23	30	27
Efficiency (min/pp)	40.6*	47.4	31.5

*Excludes ∞ values for P8, P18, P25 (zero improvement).

enabling direct implementation. This explains RAG-Assisted’s 100% success on all external integration projects (P8, P14, P18, P25) versus Self-Critique’s zero improvement.

Self-Critique Succeeded on Business Logic. Project P10 required inventory management validation logic, specifically preventing checkout when requested quantity exceeds available stock. Self-Critique’s code review identified this gap and the LLM’s critique noted “The checkout endpoint does not verify that $\text{cart.quantity} \leq \text{product.stock}$ before processing payment,” and subsequent iterations added the missing validation. This success demonstrates Self-Critique’s strength: when failures stem from incomplete or incorrect logic that is reviewable through code inspection, iterative refinement effectively identifies and fixes gaps. Notably, Multi-Model and RAG-Assisted also got 95.5% on P10, missing the inventory check. GPT-5’s design covered basic cart flow but omitted validation rules, and RAG-Assisted’s retrieved examples lacked inventory logic (often assuming unlimited stock). This indicates RAG-Assisted’s limitation: it depends on having relevant examples; without them, it cannot fill the gap.

RAG-Assisted Achieves Superior Efficiency. RAG-Assisted’s 31.5 min/pp efficiency (vs 40.6 and 47.4) comes from directly using retrieved patterns. For example, RAG-Assisted fetched Django i18n setup for P25, enabling immediate correct code. In contrast, Multi-Model adds design overhead and Self-Critique needs many review cycles or fails on missing external knowledge. Figure 1’s results confirm a bimodal outcome: Self-Critique either completes quickly (e.g. P2 in 115 min, P10 in 155 min) or yields no improvement (P8, P18, P25). Code-reviewable failures (logic failures) are solved fastest by Self-Critique, while failures needing external knowledge cannot be resolved by Self-Critique regardless of effort.

B. Decision Framework for Practitioners

Our results show that Self-Critique, Multi-Model Collaboration, and Retrieval-Augmented Generation do not compete

TABLE V
HIERARCHICAL DECISION FRAMEWORK BY FAILURE TYPE

Failure Type	1st Choice	2nd Choice	3rd Choice
Local Logic Failures	Self-Critique	RAG-A	Multi-M
External Integration Failures	RAG-A	Multi-M	Self-Critique
Domain Knowledge Failures	RAG-A	Multi-M	Self-Critique
Infrastructure Configuration Failures	RAG-A	Multi-M	Self-Critique

as mutually exclusive alternatives; instead, they attack complementary aspects of progressive prompting failures. When we apply all three enhancements, almost all remaining tasks can be recovered. However, in practice, running every enhancement on every failure is rarely desirable due to latency and cost. To make our findings actionable, we compare the methods along two axes: complete rates by failure type and time cost, and distill them into a hierarchical decision framework summarized in Table V.

The framework follows a simple principle: for each failure type, start with the fastest method that actually works reliably on that type, and escalate only when needed. For local logic failures that are visible in the code (CRUD operations, missing functions, validation rules), Self-Critique is both the fastest and the most effective. In contrast, Self-Critique shows no improvement on external integration, domain-knowledge, and infrastructure failures; in these cases, RAG-Assisted becomes the first-line choice. Multi-Model Collaboration is slower than either Self-Critique or RAG-Assisted but provides the highest reliability overall, so we position it as a second option: used when correctness is paramount or when faster methods leave residual gaps. This hierarchical view is a trade-off solution derived from our empirical data.

Practitioners can plug in their own cost constraints and failure profiles, and still reuse the same ordering principle. This decision rules can be encoded directly as a meta-prompt for LLM-based agents: given a partially implemented project and a failing task, the agent (1) classifies the failure into one of our types, (2) selects the corresponding first-line enhancement, and (3) escalates to the second and third methods only if the failure persists. In this sense, our framework not only explains when each enhancement helps, but also serves as a reusable prompt structure for orchestrating LLM tools in practice.

VI. THREATS TO VALIDITY

A. Internal Validity

We manually judged task completion (complete/partial/missing), which introduces subjectivity. To mitigate this, we used predefined checklists of required

functionality, reducing variability. Still, some edge cases between “complete” and “partial” required judgment. Our study evaluates specific LLM implementations (GPT-5, Claude Sonnet 4.5), and results may vary with different models or future versions. However, our decision framework focuses on enhancement method characteristics (iterative refinement, architectural separation, retrieval augmentation) rather than model-specific features, suggesting broader applicability across LLM platforms.

B. External Validity

Our 25 GitHub projects span a range of domains and complexity levels (3–31 tasks) and were intentionally selected to have detailed implementation instructions in their README files. This design choice improves reproducibility and allows us to measure requirement coverage in a controlled way, but it may also bias our sample toward relatively well-documented systems compared to projects with vaguer specifications.

The projects cover multiple programming languages and mainstream application domains. We therefore expect our failure taxonomy and decision framework to be most representative of common web and enterprise development settings. At the same time, the results may not fully generalize to highly specialized domains (e.g., embedded systems, scientific computing, safety-critical systems). Extending and validating the framework in these specialized contexts remains an important direction for future work.

Finally, our time-efficiency results are influenced by factors such as hardware configuration, network connectivity, and API load, so the reported wall-clock times should be interpreted as context-dependent rather than universal constants. In line with prior work on productivity and efficiency metrics in software engineering, time-based measures as *relative* indicators to compare alternatives in a given context, not as precise predictions of absolute performance [17]. In this sense, our failure-aware decision framework is designed as a simple and actionable heuristic that captures the three patterns and offers practitioners an intuitive, friendly first-step guideline for choosing an appropriate enhancement strategy.

VII. CONCLUSION AND FUTURE WORK

Our study shows that progressive prompting markedly improves requirement coverage (96.9% vs 80.5%) but still leaves some tasks unimplemented. Enhancement effectiveness varies by failure type: iterative self-critique corrects logical errors but fails on external integrations, while RAG-Assisted completes all failure types with higher efficiency. RAG-Assisted also achieved the best time-efficiency (about 31.5 minutes per percentage-point improvement). We propose a failure-driven decision framework that links each failure pattern to the most effective enhancement strategy, enabling practitioners to select methods based on project characteristics rather than trial-and-error. Our contributions include three concrete enhancement pipelines, a taxonomy of failure modes, and evidence-based guidelines for AI-assisted code generation. In future work, we plan to automate failure-type diagnosis and enhancement

selection, for example by learning classifiers or policies that can be embedded into LLM-based development agents. We also plan to validate and refine the decision framework on larger, industrial codebases.

REFERENCES

- [1] B. Wei, “Requirements are all you need: From requirements to code with llms,” in *2024 IEEE 32nd International Requirements Engineering Conference (RE)*, pp. 416–422, IEEE, 2024.
- [2] Z. Yao, A. Parashar, H. Zhou, W. S. Jang, F. Ouyang, Z. Yang, and H. Yu, “Mcqg-srefine: Multiple choice question generation and evaluation with iterative self-critique, correction, and comparison feedback,” in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 10728–10777, 2025.
- [3] Z. Dong, B. Peng, Y. Wang, J. Fu, X. Wang, X. Zhou, Y. Shan, K. Zhu, and W. Chen, “Effiqa: Efficient question-answering with strategic multi-model collaboration on knowledge graphs,” in *Proceedings of the 31st International Conference on Computational Linguistics*, pp. 7180–7194, 2025.
- [4] V. Wen, Z. Peng, and Y. Chen, “The ai imitation game: A cognitive comparison of mimicry in large language models,” in *2025 IEEE International Conference on Information Reuse and Integration and Data Science (IRI)*, pp. 79–84, IEEE, 2025.
- [5] P. Zhao, H. Zhang, Q. Yu, Z. Wang, Y. Geng, F. Fu, L. Yang, W. Zhang, J. Jiang, and B. Cui, “Retrieval-augmented generation for ai-generated content: A survey,” *arXiv preprint arXiv:2402.19473*, 2024.
- [6] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou, *et al.*, “Chain-of-thought prompting elicits reasoning in large language models,” *Advances in neural information processing systems*, vol. 35, pp. 24824–24837, 2022.
- [7] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, “Self-consistency improves chain of thought reasoning in language models,” *arXiv preprint arXiv:2203.11171*, 2022.
- [8] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, *et al.*, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” *Advances in neural information processing systems*, vol. 33, pp. 9459–9474, 2020.
- [9] S. Yao, D. Yu, J. Zhao, I. Shafraan, T. Griffiths, Y. Cao, and K. Narasimhan, “Tree of thoughts: Deliberate problem solving with large language models,” *Advances in neural information processing systems*, vol. 36, pp. 11809–11822, 2023.
- [10] M. Dahiya, R. Gill, N. Niu, H. Gudaparthi, and Z. Peng, “Leveraging chatgpt to predict requirements testability with differential in-context learning,” in *2024 IEEE International Conference on Information Reuse and Integration for Data Science (IRI)*, pp. 170–175, IEEE, 2024.
- [11] H. Ding, Z. Fan, I. Guehring, G. Gupta, W. Ha, J. Huan, L. Liu, B. Omidvar-Tehrani, S. Wang, and H. Zhou, “Reasoning and planning with large language models in code development,” in *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 6480–6490, 2024.
- [12] OpenAI, “Response from gpt-5 (model: Gpt-5-turbo, example version).” Accessed: 14 Nov. 2025, 2025. Prompt used: “Explain how to cite large language models in BibTeX.” (See Appendix A for full transcript).
- [13] J. Franco, J. Crossa, S. Taba, and H. Shands, “A sampling strategy for conserving genetic diversity when forming core subsets,” *Crop Science*, vol. 45, no. 3, pp. 1035–1044, 2005.
- [14] D. Song, Z. Zhou, Z. Wang, Y. Huang, S. Chen, B. Kou, L. Ma, and T. Zhang, “An empirical study of code generation errors made by large language models,” in *7th Annual Symposium on Machine Programming*, 2023.
- [15] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, “Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 21558–21572, 2023.
- [16] D. W. Zimmerman and B. D. Zumbo, “Relative power of the wilcoxon test, the friedman test, and repeated-measures anova on ranks,” *The Journal of Experimental Education*, vol. 62, no. 1, pp. 75–86, 1993.
- [17] C. Sadowski and T. Zimmermann, *Rethinking productivity in software engineering*. Springer Nature, 2019.